

Arsitektur dan Implementasi Pipeline Pengambilan Data Merchant Terintegrasi pada Ekosistem Layanan Pengantaran Makanan Online

Fenomena pertumbuhan ekonomi digital telah menempatkan aplikasi pengantaran makanan online sebagai salah satu sumber data paling berharga bagi analisis pasar, intelijen kompetitif, dan optimalisasi rantai pasok. Namun, upaya untuk mengekstraksi data detail merchant dalam skala besar menghadapi rintangan teknis yang sangat kompleks, terutama berkaitan dengan sistem pertahanan otomatis yang dirancang untuk menjaga ketersediaan layanan dan melindungi integritas data.¹ Dalam konteks pengambilan data merchant dari daftar yang sudah tersedia, tantangan utama bergeser dari sekadar penemuan URL (crawling) menjadi stabilitas pengambilan data (scraping) yang harus mampu menembus mekanisme proteksi seperti status kode 403 Forbidden, pembatasan laju (rate limiting), dan deteksi perilaku bot.² Laporan ini akan menguraikan secara komprehensif arsitektur pipeline pengambilan data yang tangguh, menggunakan ekosistem Node.js, dengan fokus pada keberhasilan tingkat tinggi, penanganan batch, dan mekanisme pemulihan otomatis.

Dinamika Pertahanan Anti-Bot dan Analisis Kegagalan Akses

Kegagalan pengambilan data sering kali bermanifestasi dalam bentuk status kode 403 Forbidden, yang menandakan bahwa server memahami permintaan tersebut tetapi menolak untuk memberikan otorisasi karena terdeteksi sebagai lalu lintas non-manusia.¹ Di tahun 2024 dan 2025, platform pengantaran makanan besar telah mengadopsi Web Application Firewalls (WAF) canggih seperti Cloudflare, DataDome, dan Akamai yang menggunakan analisis perilaku berbasis pembelajaran mesin untuk mengidentifikasi skrip otomatis.¹ Analisis mendalam menunjukkan bahwa deteksi ini tidak lagi hanya mengandalkan alamat IP, melainkan pada kombinasi tanda sidik jari (fingerprinting) di berbagai lapisan protokol.³

Lapisan Deteksi dan Sidik Jari TLS

Salah satu alasan paling umum mengapa skrip Node.js gagal sementara peramban web biasa berhasil adalah perbedaan pada sidik jari Transport Layer Security (TLS).³ Setiap klien HTTP memiliki cara unik dalam melakukan jabat tangan (handshake) TLS, termasuk urutan cipher suite yang didukung dan ekstensi yang dikirimkan.³ Sistem anti-bot modern menggunakan algoritma hashing JA3 atau JA4 untuk memetakan karakteristik koneksi ini.³ Jika sebuah permintaan menggunakan User-Agent yang mengaku sebagai Chrome versi terbaru tetapi

mengirimkan jabat tangan TLS yang khas dari pustaka internal Node.js, sistem proteksi akan segera memicu blokir 403.³

Komponen Deteksi	Metode Analisis	Dampak pada Pipeline
IP Reputation	Pengecekan terhadap database pusat data vs perumahan. ⁵	Penggunaan IP datacenter sering kali memicu blokir instan pada target sensitif. ⁵
TLS Fingerprinting	Analisis JA3/JA4 pada saat handshake TLS berlangsung. ³	Ketidakesesuaian antara User-Agent dan sidik jari protokol mengakibatkan blokir 403. ³
HTTP Headers	Verifikasi keberadaan dan urutan header Sec-Fetch-*. ³	Header yang tidak lengkap atau tidak konsisten menandakan permintaan otomatis. ³
Behavioral	Analisis pola waktu antar permintaan dan urutan navigasi. ⁵	Permintaan yang terlalu konsisten atau berurutan secara linier mudah dikenali sebagai bot. ⁵

Kegagalan dalam menangani lapisan-lapisan ini secara holistik akan menyebabkan penurunan drastis pada success rate pipeline, terutama ketika volume data yang diambil meningkat melampaui ambang batas normal.¹ Oleh karena itu, strategi bypass yang efektif harus mencakup rotasi identitas yang kredibel dan emulasi perilaku yang menyerupai interaksi manusia secara organik.¹

Pemilihan Infrastruktur Perangkat Lunak pada Ekosistem Node.js

Node.js dipilih sebagai basis teknologi karena model I/O non-blocking yang sangat efisien untuk tugas-tugas yang didominasi oleh operasi jaringan, seperti pengambilan data dari API pihak ketiga.⁹ Dalam membangun pipeline yang stabil, pemilihan pustaka HTTP dan pengelola antrean (queue manager) menjadi keputusan arsitektural yang paling krusial.

Perbandingan Pustaka HTTP: Got vs. Undici vs. Axios

Meskipun Axios sangat populer dalam pengembangan web umum, pustaka tersebut sering kali memerlukan banyak konfigurasi tambahan atau plugin pihak ketiga untuk menangani retries

dan rotasi proxy secara otomatis.¹² Sebaliknya, pustaka Got dirancang khusus untuk skenario pengambilan data yang tangguh, menyediakan dukungan bawaan untuk HTTP/2, pembatalan permintaan, dan sistem hook yang sangat fleksibel.¹² Di sisi lain, Undici menawarkan performa murni yang lebih tinggi karena merupakan klien HTTP yang ditulis ulang untuk inti Node.js, namun memiliki abstraksi yang lebih rendah sehingga memerlukan implementasi manual untuk logika retry yang kompleks.¹⁶

Untuk pipeline yang memprioritaskan stabilitas dan kemudahan penanganan error, Got sering kali menjadi pilihan utama karena fitur retry yang sangat canggih dan kemampuan untuk menyesuaikan perilaku berdasarkan kode status HTTP tertentu.¹⁵

Manajemen Konkurensi dengan p-queue dan Bottleneck

Mengambil data dalam jumlah besar memerlukan kontrol yang ketat terhadap jumlah permintaan simultan agar tidak membebani infrastruktur target atau memicu mekanisme pertahanan rate limiting.¹⁸ p-queue menyediakan solusi berbasis janji (promise-based) yang memungkinkan pengaturan batas konkurensi, prioritas tugas, dan kemampuan untuk menjeda (pause) eksekusi jika terjadi error sistemik seperti 403.¹⁸ Sementara itu, Bottleneck menawarkan kontrol yang lebih halus terhadap waktu (time-based rate limiting), yang sangat berguna untuk mematuhi aturan "X permintaan per detik" yang sering diterapkan oleh API aplikasi pengantaran makanan.¹⁹

Kombinasi yang ideal untuk pipeline skala besar adalah menggunakan p-queue untuk manajemen tugas batch dan Bottleneck untuk memastikan distribusi permintaan yang merata sepanjang waktu, sehingga menghindari lonjakan lalu lintas yang mencurigakan.¹⁸

Mekanisme Ketangguhan: Retry, Backoff, dan Jitter

Dalam lingkungan jaringan yang tidak stabil, kegagalan adalah sebuah kepastian. Pipeline yang sukses tidak hanya mencoba kembali permintaan yang gagal, tetapi melakukannya dengan strategi yang meminimalkan risiko pemblokiran lebih lanjut dan memaksimalkan peluang pemulihan.²³

Strategi Exponential Backoff

Mekanisme retry yang paling efektif menggunakan algoritma exponential backoff, di mana waktu tunggu antara upaya retry meningkat secara eksponensial.²⁴ Hal ini memberikan waktu bagi server target untuk pulih dari beban berlebih atau bagi sistem proteksi untuk menurunkan tingkat kecurigaan terhadap alamat IP tertentu.²⁴ Rumus dasar untuk menghitung waktu tunda (d) berdasarkan jumlah percobaan (n) dan tunda dasar (b) adalah:

$$d = b \cdot 2^{n-1}$$

Implementasi yang lebih canggih harus menyertakan "jitter", yaitu variasi acak pada waktu tunda untuk mencegah fenomena "thundering herd", di mana banyak pekerja (workers) melakukan retry secara bersamaan pada waktu yang sama, yang justru dapat memperburuk kondisi server target.²⁴

Parameter Retry	Rekomendasi	Alasan Arsitektural
Max Attempts	3 - 5 kali	Menghindari loop tak terbatas pada error permanen. ²⁹
Base Delay	1.000 ms - 2.000 ms	Memberikan jeda awal yang cukup untuk kegagalan transien. ³⁰
Max Delay	30.000 ms - 60.000 ms	Membatasi waktu tunda agar pipeline tetap produktif. ²⁴
Jitter Factor	0,1 - 0,3 (10% - 30%)	Mendisinkronisasi upaya retry dari berbagai worker. ²⁴

Diferensiasi Penanganan Error

Pipeline harus mampu membedakan antara error yang layak dicoba kembali dan error yang memerlukan intervensi atau perubahan strategi.²⁴ Status 429 (Too Many Requests) harus memicu penundaan yang lebih lama atau rotasi alamat IP, sementara status 404 (Not Found) harus dicatat sebagai kegagalan permanen untuk menghindari pemborosan sumber daya.³ Khusus untuk status 403, pipeline harus segera mengevaluasi apakah sidik jari browser saat ini masih valid atau sudah masuk dalam daftar blokir sistem WAF.¹

Arsitektur Batching dan Manajemen Memori

Mengambil data merchant dalam skala jutaan memerlukan pendekatan pemrosesan batch untuk menjaga stabilitas penggunaan memori (RAM) pada Node.js.³² Memasukkan seluruh daftar merchant ke dalam memori sekaligus akan menyebabkan lonjakan penggunaan heap yang dapat memicu crash proses atau memperlambat eksekusi karena siklus garbage collection yang terlalu sering.³⁵

Pemrosesan Berbasis Aliran (Streaming)

Alih-alih membaca seluruh daftar merchant dari database atau file JSON ke dalam array, pipeline harus menggunakan aliran data (streams) atau generator asinkron.³² Dengan cara ini,

data dibaca dan diproses dalam unit-unit kecil. Strategi batching yang umum adalah mengambil sejumlah N merchant (misalnya 100), memprosesnya secara paralel hingga konkurensi maksimal tercapai, dan hanya mengambil batch berikutnya setelah batch saat ini mendekati penyelesaian.³³

Stabilitas Heap Memori pada Node.js

Node.js mengalokasikan memori dalam heap yang terbagi menjadi "New Space" untuk objek berumur pendek dan "Old Space" untuk data yang persisten.³⁶ Dalam pengambilan data besar, representasi JSON dari detail merchant dapat menumpuk di Old Space jika tidak dikelola dengan baik. Pengaturan flag `--max-old-space-size` pada saat menjalankan skrip dapat membantu memberikan ruang bernapas bagi pipeline, namun optimasi kode melalui penggunaan stream tetap menjadi solusi jangka panjang yang paling stabil.³²

Implementasi Kode: Pipeline Pengambil Data Merchant

Berikut adalah implementasi praktis menggunakan pustaka Got untuk HTTP, p-queue untuk manajemen batch/konkurensi, dan Better-SQLite3 untuk persistensi status pengambilan data. Kode ini dirancang untuk memenuhi semua persyaratan: retry, backoff, logging, dan penanganan batch.

JavaScript

```
import got from 'got';
import PQueue from 'p-queue';
import Database from 'better-sqlite3';
import winston from 'winston';
import fs from 'fs';

/**
 * KONFIGURASI PIPELINE
 */
const CONFIG = {
  concurrency: 5, // Jumlah permintaan paralel
  maxRetries: 3, // Maksimal percobaan ulang per merchant
  initialDelay: 2000, // Delay dasar (ms)
  batchSize: 100, // Ukuran pengambilan data per siklus
  timeoutLimit: 15000, // Timeout permintaan (ms)
  dbPath: './merchant_pipeline.db',
```

```

    outputPath: './output_merchants.json'
  };

  /**
   * SISTEM LOGGING TERSTRUKTUR
   * Mencatat setiap kejadian sukses/gagal per merchant [37, 38]
   */
  const logger = winston.createLogger({
    level: 'info',
    format: winston.format.combine(
      winston.format.timestamp(),
      winston.format.json()
    ),
    transports: [
      new winston.transports.File({ filename: 'pipeline_error.log', level: 'error' }),
      new winston.transports.File({ filename: 'pipeline_success.log', level: 'info' }),
      new winston.transports.Console({ format: winston.format.simple() })
    ]
  });

  /**
   * PERSISTENSI DAN MANAJEMEN STATE
   * Menggunakan SQLite untuk resume otomatis jika proses terhenti [39, 40]
   */
  const db = new Database(CONFIG.dbPath);
  db.exec(`
    CREATE TABLE IF NOT EXISTS queue (
      id TEXT PRIMARY KEY,
      status TEXT DEFAULT 'pending',
      retry_count INTEGER DEFAULT 0,
      last_error TEXT,
      last_attempt TIMESTAMP
    )
  `);

  /**
   * FUNGSI UTAMA PENGAMBILAN DATA
   * Implementasi retry, backoff, dan error handling khusus [15, 24]
   */
  const fetchMerchantDetail = async (merchantId) => {
    // Simulasi rotasi header agar menyerupai browser asli
    const headers = {
      'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/121.0.0.0 Safari/537.36',
      'Accept': 'application/json',

```

```

    'Accept-Language': 'id-ID,id;q=0.9,en-US;q=0.8,en;q=0.7',
    'Sec-Fetch-Mode': 'cors',
    'Sec-Fetch-Site': 'same-origin',
    'Sec-Fetch-Dest': 'empty'
  };

  try {
    const url = `https://api.food-app.com/v1/merchants/${merchantId}`;

    const response = await got(url, {
      headers,
      timeout: { request: CONFIG.timeoutLimit },
      retry: {
        limit: CONFIG.maxRetries,
        // Implementasi exponential backoff dengan jitter [15, 31]
        calculateDelay: ({ attemptCount, error, computedValue }) => {
          if (error.response?.statusCode === 403) return 0; // Jangan retry otomatis jika forbidden
          keras
          const delay = Math.min(CONFIG.initialDelay * Math.pow(2, attemptCount - 1),
30000);
          const jitter = Math.random() * 1000;
          return delay + jitter;
        }
      }
    }).json();

    // Logging Sukses
    logger.info({ merchantId, status: 'success', timestamp: new Date() });

    // Update DB State
    db.prepare("UPDATE queue SET status = 'completed', last_attempt = CURRENT_TIMESTAMP
WHERE id =?").run(merchantId);

    return { id: merchantId, data: response, success: true };
  } catch (error) {
    const reason = error.response? `HTTP ${error.response.statusCode}` : error.code;

    // Logging Gagal
    logger.error({ merchantId, status: 'failed', reason, timestamp: new Date() });

    // Update DB State dengan Error
    db.prepare("UPDATE queue SET status = 'failed', last_error =?, last_attempt =
CURRENT_TIMESTAMP WHERE id =?")

```

```

        .run(reason, merchantId);

    if (error.response?.statusCode === 403) {
        // Logika khusus jika terdeteksi bot (Forbidden)
        // Bisa memicu pause pada queue jika diperlukan [21]
    }

    return { id: merchantId, success: false, reason };
}
};

/**
 * ORKESTRASI BATCH PROCESSING
 * Menangani pengambilan data dalam jumlah besar secara bertahap [33, 34]
 */
const runPipeline = async () => {
    const queue = new PQueue({ concurrency: CONFIG.concurrency });
    const results = [];

    // Ambil daftar merchant yang masih 'pending' dari database
    const merchants = db.prepare("SELECT id FROM queue WHERE status IN ('pending', 'failed') AND
retry_count < ?")
        .all(CONFIG.maxRetries);

    logger.info(`Memulai pipeline untuk ${merchants.length} merchant.`);

    for (const merchant of merchants) {
        queue.add(async () => {
            const result = await fetchMerchantDetail(merchant.id);
            if (result.success) {
                // Simpan hasil secara inkremental ke file untuk efisiensi memori
                fs.appendFileSync(CONFIG.outputPath, JSON.stringify(result.data) + ",\n");
            }
        });
    }

    // Mekanisme backpressure untuk mencegah antrean memori yang berlebihan
    if (queue.size > CONFIG.batchSize) {
        await queue.onSizeLessThan(CONFIG.batchSize / 2);
    }
}

await queue.onIdle();
logger.info("Pipeline selesai memproses seluruh batch.");

```



```
};
```

```
// Eksekusi Pipeline  
// runPipeline().catch(err => logger.error("Fatal Pipeline Error", err));
```

Strategi Bypass Proteksi Tingkat Lanjut

Membangun pipeline yang "tanpa error" dalam skala besar memerlukan adaptasi terhadap sistem proteksi yang semakin cerdas. Jika teknik header statis mulai menghasilkan kegagalan, pipeline harus ditingkatkan ke teknik emulasi yang lebih mendalam.¹

Rotasi Proxy dan Reputasi IP

Sistem anti-bot pada aplikasi pengantaran makanan sering kali memantau reputasi IP. IP dari pusat data (datacenter) memiliki skor kepercayaan yang rendah karena jarang digunakan oleh pengguna aplikasi asli.⁵ Untuk mencapai success rate tinggi, pipeline harus mengintegrasikan rotasi proxy residensial.⁵ Proxy residensial menggunakan alamat IP dari penyedia layanan internet (ISP) rumahan, sehingga permintaan tampak berasal dari pelanggan nyata. Selain itu, penggunaan proxy seluler (mobile proxies) menawarkan tingkat keberhasilan tertinggi karena alamat IP seluler sering kali dibagikan oleh banyak pengguna (melalui CGNAT), sehingga sistem anti-bot sangat ragu untuk memblokir IP tersebut secara total.⁶

Emulasi Header "Sec-Fetch" dan Konteks Permintaan

Header HTTP modern memainkan peran penting dalam membedakan bot. Pipeline harus menyertakan header dari keluarga Sec-Fetch-* yang konsisten dengan jenis permintaan yang dilakukan.³

- Sec-Fetch-Dest: Menggambarkan tujuan permintaan (misalnya empty untuk panggilan API).
- Sec-Fetch-Mode: Menggambarkan mode permintaan (misalnya cors).
- Sec-Fetch-Site: Hubungan antara inisiator dan target (misalnya same-origin).

Ketidakkonsistenan antara header ini (misalnya mengaku sebagai permintaan same-origin tetapi dikirimkan tanpa header Referer yang valid) akan segera memicu deteksi pada firewall modern seperti Cloudflare atau DataDome.³

Header	Nilai Contoh	Relevansi Bot Detection
sec-ch-ua	"Not A(Brand";v="99", "Google Chrome";v="121"	Menunjukkan versi mesin peramban yang spesifik. ³

sec-ch-ua-mobile	?0	Menandakan apakah klien adalah perangkat seluler. ³
sec-ch-ua-platform	"Windows"	Harus cocok dengan User-Agent yang dikirimkan. ³

Optimalisasi Performa dan Keandalan Jangka Panjang

Pipeline yang stabil harus mampu berjalan selama sehari-hari tanpa intervensi manual. Hal ini memerlukan sistem monitoring dan manajemen state yang kuat.¹

Logging Terperinci dan Auditability

Setiap permintaan dalam pipeline harus menghasilkan log yang mencakup ID merchant, durasi permintaan, alamat IP proxy yang digunakan, dan hasil akhirnya (sukses/gagal beserta alasannya).³⁷ Logging terperinci memungkinkan tim pengembang untuk melakukan analisis statistik terhadap kegagalan. Jika ditemukan bahwa kegagalan terkonsentrasi pada wilayah proxy tertentu atau jam-jam tertentu, strategi rotasi dapat disesuaikan secara dinamis.¹

Skema Data JSON Terstruktur

Data merchant yang diambil harus disimpan dalam format JSON yang terstruktur dan divalidasi.⁴³ Penggunaan standar seperti schema.org/FoodEstablishment sangat disarankan untuk memastikan data dapat dikonsumsi oleh berbagai aplikasi hilir tanpa perlu transformasi tambahan.⁴⁵

Contoh struktur output JSON yang dioptimalkan untuk industri makanan:

JSON

```
{
  "merchant_id": "ID_12345",
  "name": "Restoran Bintang Lima",
  "cuisine": ["Indonesian", "Asian"],
  "location": {
    "latitude": -6.2088,
    "longitude": 106.8456,
    "address": "Jl. Sudirman No. 1, Jakarta"
  },
  "rating": 4.8,
  "opening_hours": "Mo-Su 08:00-22:00",
}
```

```
"menu_summary": {  
  "item_count": 45,  
  "price_range": "IDR 50.000 - 150.000"  
},  
"metadata": {  
  "scraped_at": "2025-04-24T06:46:00Z",  
  "source_app": "FoodDeliveryApp_X"  
}  
}
```

43

Manajemen Transaksi dan Persistensi dengan SQLite

Penggunaan database ringan seperti SQLite (melalui better-sqlite3) jauh lebih unggul dibandingkan menyimpan status dalam file teks atau memori sederhana.³⁹ SQLite mendukung transaksi ACID, yang menjamin bahwa jika sistem mati secara mendadak, status "completed" untuk seorang merchant tidak akan hilang atau rusak.⁴⁰ Selain itu, SQLite memungkinkan pengambilan batch merchant dengan kueri SQL yang efisien, memudahkan implementasi fitur "resume from last successful item".⁴⁰

Strategi Penanganan Error Lanjutan (Advanced Error Handling)

Selain retry sederhana, pipeline harus menerapkan pola "Circuit Breaker".²⁷ Jika tingkat kegagalan (misalnya status 403) melebihi ambang batas tertentu dalam jendela waktu yang singkat, pipeline harus secara otomatis menghentikan operasi atau menurunkan kecepatan secara drastis. Ini mencegah "pembakaran" seluruh pool IP proxy ketika sistem proteksi target sedang melakukan pengetatan keamanan secara global.⁴⁹

Analisis Header "Retry-After"

Banyak API aplikasi pengantaran makanan yang canggih akan mengirimkan header Retry-After saat mengembalikan status 429.³ Pipeline harus secara eksplisit menghormati nilai ini daripada menggunakan tunda acak. Jika server meminta untuk menunggu selama 60 detik, melakukan percobaan kembali dalam 2 detik hanya akan memperburuk reputasi alamat IP di mata firewall target.²⁴

Penanganan Timeout yang Adaptif

Timeout sering terjadi pada pengambilan data skala besar karena kemacetan jaringan atau beban server target yang berfluktuasi.² Pipeline harus menerapkan dua jenis timeout:

1. **Connection Timeout:** Berapa lama menunggu untuk membangun koneksi awal.

2. **Read Timeout:** Berapa lama menunggu data setelah koneksi terjalin.

Nilai timeout yang terlalu pendek akan menyebabkan banyak kegagalan palsu pada koneksi proxy yang lambat, sementara nilai yang terlalu lama akan menghambat throughput pipeline secara keseluruhan.³⁴

Kesimpulan dan Rekomendasi Arsitektur

Membangun pipeline pengambilan data merchant yang sukses memerlukan keseimbangan antara efisiensi eksekusi dan tingkat penyamaran perilaku.¹ Penggunaan Node.js dengan pustaka seperti Got dan p-queue memberikan fondasi yang kuat untuk menangani ribuan permintaan secara asinkron tanpa membebani sistem.⁹ Namun, kunci utama untuk menghindari blokir 403 Forbidden terletak pada kemampuan pipeline untuk mengemulasi sidik jari browser secara akurat dan menggunakan infrastruktur proxy yang memiliki reputasi tinggi.³

Rekomendasi operasional untuk skalabilitas tinggi:

1. **Gunakan Proxy Residensial atau Seluler:** Untuk meminimalkan deteksi berbasis reputasi IP.⁵
2. **Emulasi Sidik Jari TLS:** Pastikan pustaka HTTP yang digunakan mendukung penyesuaian cipher suite agar cocok dengan User-Agent.³
3. **Persistensi State Inkremental:** Selalu simpan status setiap permintaan ke database lokal untuk memungkinkan pemulihan pasca kegagalan.³⁹
4. **Monitoring Sukses/Gagal Real-time:** Implementasikan dashboard sederhana atau peringatan otomatis jika tingkat keberhasilan turun di bawah 95%.¹

Dengan mengikuti prinsip-prinsip arsitektur ini, pipeline pengambilan data dapat beroperasi dengan tingkat stabilitas yang tinggi, memberikan aliran data merchant yang bersih dan terstruktur untuk mendukung pengambilan keputusan bisnis yang krusial.¹

Works cited

1. Solving 403 Errors in Web Scraping: The Ultimate Guide for 2025 ..., accessed on April 24, 2026, <https://rebrowser.net/blog/solving-403-errors-in-web-scraping-the-ultimate-guide-or-bypass-protection-successfully>
2. Here's Why You're Getting 403 in Python Requests and How to Fix It | Scrape.do, accessed on April 24, 2026, <https://scrape.do/blog/python-requests-403-forbidden/>
3. How to Fix 403 Forbidden Errors When Web Scraping - Scrapfly Blog, accessed on April 24, 2026, <https://scrapfly.io/blog/posts/403-forbidden-web-scraping>
4. Anti-Bot Bypass That Actually Works - Scrape.do, accessed on April 24, 2026, <https://scrape.do/features/anti-bot-bypass/>
5. Bypass Bot Detection: 5 Best Methods - ZenRows, accessed on April 24, 2026, <https://www.zenrows.com/blog/bypass-bot-detection>
6. How to Bypass Anti-Bot Protection When Web Scraping - Scrapfly Blog, accessed

on April 24, 2026,

<https://scrapfly.io/blog/posts/how-to-bypass-anti-bot-protection-when-web-scraping>

7. Web Scraping: How to Solve 403 Errors - Scrapeless, accessed on April 24, 2026, <https://www.scrapeless.com/en/blog/how-to-solve-403-errors>
8. Web Scraping without getting blocked (2026 Solutions) - ScrapingBee, accessed on April 24, 2026, <https://www.scrapingbee.com/blog/web-scraping-without-getting-blocked/>
9. Mastering Web Scraping with Node.js: A Beginner's Guide - Mindpathtech, accessed on April 24, 2026, <https://www.mindpathtech.com/blog/web-scraping-with-nodejs-guide/>
10. Node.js Web Scraping Tutorial: Complete Guide with Code Examples 2026 | ScrapeBadger, accessed on April 24, 2026, <https://scrapebadger.com/blog/nodejs-web-scraping-tutorial-complete-guide-with-code-examples-2026>
11. Building Reliable Data Scrapers: A Practical Guide to Web Scraping with JavaScript and Node.js | by DataDwip | Medium, accessed on April 24, 2026, <https://medium.com/@datadwipscraping/building-reliable-data-scrapers-a-practical-guide-to-web-scraping-with-javascript-and-node-js-2083f42e3742>
12. 5 Node.js HTTP Request Methods Every Developer Should Master - Carmatec, accessed on April 24, 2026, <https://www.carmatec.com/blog/5-nodejs-http-request-methods/>
13. Axios Alternatives: Exploring Top HTTP Clients in 2025 - Scrapeless, accessed on April 24, 2026, <https://www.scrapeless.com/en/blog/axios-alternatives>
14. How to Solve Axios 403 Forbidden Error - ZenRows, accessed on April 24, 2026, <https://www.zenrows.com/blog/axios-403>
15. got/documentation/7-retry.md at main · sindresorhus/got · GitHub, accessed on April 24, 2026, <https://github.com/sindresorhus/got/blob/main/documentation/7-retry.md>
16. How to Integrate Undici with ZenRows, accessed on April 24, 2026, <https://docs.zenrows.com/integrations/undici>
17. try undici http client · Issue #1419 · sindresorhus/got - GitHub, accessed on April 24, 2026, <https://github.com/sindresorhus/got/issues/1419>
18. p-limit vs p-queue vs p-throttle | Concurrency and Rate Limiting in ..., accessed on April 24, 2026, <https://npm-compare.com/p-throttle,p-limit,p-queue>
19. p-limit vs p-queue vs p-throttle | Concurrency and Rate Limiting in JavaScript, accessed on April 24, 2026, <https://npm-compare.com/p-limit,p-queue,p-throttle>
20. p-queue@9.1.2 - jsDocs.io, accessed on April 24, 2026, <https://www.jsdocs.io/package/p-queue>
21. p-queue - npm, accessed on April 24, 2026, <https://www.npmjs.com/package/p-queue>
22. SGrondin/bottleneck: Job scheduler and rate limiter, supports Clustering - GitHub, accessed on April 24, 2026, <https://github.com/SGrondin/bottleneck>
23. (New) Best Node.js Retry Library for Async Functions (2026) - DEV Community, accessed on April 24, 2026, <https://dev.to/darshan1005/retry-pro-vs-p-retry-best-nodejs-retry-library-for-async-f>

[unctions-2026-4kg4](#)

24. Building Resilient Node.js Services with Exponential Backoff | by Nasik Nazzar - Medium, accessed on April 24, 2026, <https://medium.com/@mnnasik7/building-resilient-node-js-services-with-exponential-backoff-5334fa5a3f7e>
25. Building a Robust Web Scraper with Error Handling and Retry Logic - Medium, accessed on April 24, 2026, <https://medium.com/techtrends-digest/building-a-robust-web-scraper-with-error-handling-and-retry-logic-3e7b6541bbbc>
26. Stop Breaking Your APIs - How to Implement Proper Retry and Exponential Backoff in NestJS | Jean-Marc Möckel, accessed on April 24, 2026, <https://jean-marc.io/blog/stop-breaking-your-apis-how-to-implement-proper-retry-and-exponential-backoff-in-nestjs>
27. How to Implement Retry Logic in Go with Exponential Backoff - OneUptime, accessed on April 24, 2026, <https://oneuptime.com/blog/post/2026-01-07-go-retry-exponential-backoff/view>
28. Retry pattern: Implementing Exponential Backoff | by Shakyaprabeg - Medium, accessed on April 24, 2026, <https://medium.com/@shakyaprabeg/retry-pattern-implementing-exponential-backoff-8ce38387f443>
29. Best practice of retry strategy | DoorDash Developer Services, accessed on April 24, 2026, https://developer.doordash.com/en-US/docs/drive/reference/retry_pattern/
30. How to Implement Retry with Exponential Backoff in Node.js - CodeWithYou, accessed on April 24, 2026, <https://www.codewithyou.com/blog/how-to-implement-retry-with-exponential-backoff-in-nodejs>
31. retry-axios - Axios Retry Interceptor with Exponential Backoff | Justin Beckwith, accessed on April 24, 2026, <https://jbeckwith.com/projects/retry-axios>
32. Handling Large Data in Node.js: Performance Tips & Best Practices - DEV Community, accessed on April 24, 2026, <https://dev.to/saikumar2121/handling-large-data-in-nodejs-performance-tips-best-practices-29of>
33. How to perform Node.js batch processing - Educative.io, accessed on April 24, 2026, <https://www.educative.io/answers/how-to-perform-nodejs-batch-processing>
34. How to Process Large Datasets with Parallel Jobs in Node.js - OneUptime, accessed on April 24, 2026, <https://oneuptime.com/blog/post/2026-01-25-parallel-jobs-large-datasets-nodejs/view>
35. Processing 1 Million Records in Node.js and MySQL Efficiently - DEV Community, accessed on April 24, 2026, <https://dev.to/pawandeore/processing-1-million-records-in-nodejs-and-mysql-efficiently-ca2>
36. Understanding and Tuning Memory | Node.js Learn, accessed on April 24, 2026, <https://nodejs.org/learn/diagnostics/memory/understanding-and-tuning-memory>

37. Mastering Winston for Production Logging in Node.js - Dash0, accessed on April 24, 2026, <https://www.dash0.com/guides/winston-production-logging-nodejs>
38. Understanding Better-SQLite3: The Fastest SQLite Library for Node.js - DEV Community, accessed on April 24, 2026, <https://dev.to/lovestaco/understanding-better-sqlite3-the-fastest-sqlite-library-for-nodejs-4n8>
39. Convince me to use better-sqlite3 #262 - GitHub, accessed on April 24, 2026, <https://github.com/WiseLibs/better-sqlite3/issues/262>
40. Real-time Data Processing with Node.js: Techniques and Best Practices - Dev3lop, accessed on April 24, 2026, <https://dev3lop.com/real-time-data-processing-with-node-js-techniques-and-best-practices/>
41. Best Practices for Logging in Node.js - AppSignal Blog, accessed on April 24, 2026, <https://blog.appsignal.com/2021/09/01/best-practices-for-logging-in-nodejs.html>
42. restaurants.json - xujiahaha/food-delivery-app - GitHub, accessed on April 24, 2026, <https://github.com/xujiahaha/food-delivery-app/blob/master/restaurants.json>
43. How to Build Scraping Skills for AI Agents: Incremental Runs, Stop Conditions, and Parallel Execution | MindStudio, accessed on April 24, 2026, <https://www.mindstudio.ai/blog/build-scraping-skills-ai-agents-incremental-parallel>
44. FoodEstablishment - Schema.org Type, accessed on April 24, 2026, <https://schema.org/FoodEstablishment>
45. Product Schema Markup: Complete Implementation Guide for Ecommerce Success, accessed on April 24, 2026, <https://hashmeta.com/blog/product-schema-markup-complete-implementation-guide-for-ecommerce-success/>
46. Set up structured data for Merchant Center - Google Help, accessed on April 24, 2026, <https://support.google.com/merchants/answer/7331077?hl=en>
47. Ultimate guide: Best databases for NodeJS apps - Ably, accessed on April 24, 2026, <https://ably.com/blog/best-nodejs-databases>
48. Best Practices to Engineering Big Data Pipeline Architecture - GroupBWT, accessed on April 24, 2026, <https://groupbwt.com/blog/big-data-pipeline-architecture/>
49. Anti-scraping protections | Academy - Apify Documentation, accessed on April 24, 2026, <https://docs.apify.com/academy/anti-scraping>
50. enigmamarketing/batch-api-requests: Nodejs Client and Express middleware for handling batch HTTP requests. - GitHub, accessed on April 24, 2026, <https://github.com/enigmamarketing/batch-api-requests/>